Name	AYUSHI JAPSARE
UID no.	2023301006
EXP	4

CSS	
AIM:	To implement the Diffie Hellman (DH) key exchange protocol to allow two parties to securely agree on a shared secret over an insecure channel. (i) selection of public parameters (prime p, generator g) (ii) private/public key generation for two parties (iii) computation of the shared secret.
PROBLEM STATEMENT:	Security of DH relies on the intractability of solving the discrete logarithm in the chosen group. In practice, safe primes p (where p = 2q + 1 and q is prime) and generators of the order-q subgroup are recommended to mitigate small subgroup attacks. The shared secret K is not used directly; instead, a Key Derivation Function (KDF) such as SHA-256 is applied to produce a fixed-length symmetric key. Algorithm / Steps – 1. Choose a large prime p and a generator g of a large prime-order subgroup modulo p (or use a standardized group). 2. Alice selects a random private key a in [2, p−2] and computes A = g^a mod p. 3. Bob selects a random private key b in [2, p−2] and computes B = g^b mod p. 4. Alice sends A to Bob; Bob sends B to Alice over the public channel. 5. Alice computes the shared secret K_A = B^a mod p; Bob computes K_B = A^b mod p. 6. Since K_A = K_B = g^(ab) mod p, both sides now share K. Apply KDF = SHA-256(K) to derive a symmetric key
THEORY	**1. Role of *p* and *g*** ● *p* is a large prime number that defines the finite cyclic group in which computations are performed. ● *g* is the generator (or primitive root modulo *p*) that ensures all elements of the group can be reached through exponentiation. ● Together, *(p, g)* form the **public parameters** that can be safely shared without affecting security.

	2. Choice of Private Exponents (a, b) ● Alice and Bob each choose private exponents (*a* and *b*) randomly from the range [2,p−2][2, p-2][2,p−2]. ● These values must remain secret, as they are the **basis of security** in the protocol. ● Even if the public values (*A = g^a mod p, B = g^b mod p*) are exposed, recovering *a* or *b* is computationally infeasible due to the **Discrete Logarithm Problem (DLP)**. **3. Discrete Log Hardness** ● The security of Diffie–Hellman relies on the fact that computing gamod pg^a \mod pgamodp is easy, but recovering *a* from gamod pg^a \mod pgamodp is computationally infeasible for large primes. ● This hardness is what prevents attackers from deriving the private keys or the shared secret even if public values are intercepted. **4. Key Derivation Function (KDF) Usage** ● The raw shared secret K=gabmod pK = g^{ab} \mod pK=gabmodp may not be uniformly random or suitable for direct use as a symmetric key. ● A **KDF**, such as SHA-256, is applied to *K* to produce a fixed-length cryptographic key. ● This ensures the final key is secure, pseudorandom, and compatible with symmetric encryption algorithms. **5. Attack Considerations** ● **Man-in-the-Middle (MITM):** If authentication is not used, an attacker can intercept and replace public values (*A, B*), establishing separate secrets with Alice and Bob. Mitigation: use digital signatures or certificates. ● **Small Subgroup Attacks:** If *g* does not generate a large prime-order subgroup, attackers may force key negotiation into a small subgroup where the secret can be brute-forced. Mitigation: use *safe primes* (*p = 2q + 1*, with *q* prime) and verify subgroup order.

	• **Replay and Parameter Injection Attacks:** Avoid reusing weak or non-standard parameters; rely on standardized, vetted primes and generators.

CODE:	import secrets
	import hashlib
	import sys
	import textwrap
	from typing import Tuple, Optional
	
	def _miller_rabin_witness(a: int, d: int, n: int, r: int) -> bool:
	  x = pow(a, d, n)
	  if x == 1 or x == n - 1:
	  return False
	  for _ in range(r - 1):
	  x = (x * x) % n
	  if x == n - 1:
	  return False
	  return True # composite
	
	def is_probable_prime(n: int, rounds: int = 32) -> bool:
	  if n < 2:
	  return False
	  small_primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
	  for p in small_primes:
	  if n % p == 0:
	  return n == p
	  d = n - 1
	  r = 0
	  while d % 2 == 0:
	  r += 1
	  d //= 2
	  for _ in range(rounds):
	  a = secrets.randbelow(n - 3) + 2
	  if _miller_rabin_witness(a, d, n, r):
	  return False
	  return True
	
	def gen_prime(bits: int) -> int:
	  while True:

```
        candidate = secrets.randbits(bits) | (1 << (bits - 1)) | 1
        if is_probable_prime(candidate):
            return candidate

def gen_safe_prime(bits: int) -> Tuple[int, int]:
    while True:
        q = gen_prime(bits - 1)
        p = 2 * q + 1
        if is_probable_prime(p):
            return p, q

def gen_subgroup_generator(p: int, q: int) -> int:
    while True:
        h = secrets.randbelow(p - 3) + 2
        g = pow(h, 2, p)
        if g != 1 and pow(g, q, p) == 1:
            return g

RFC3526_GROUP14_P_HEX = """
FFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD129024E08
8A67CC74020BBEA63B139B22514A08798E3404DDEF9519B3CD3A431B
302B0A6DF25F14374FE1356D6D51C245E485B576625E7EC6F44C42E9
A63A3620FFFFFFFFFFFFFFFF
""".replace("\n", "").replace(" ", "")

def get_rfc3526_group14() -> Tuple[int, int]:
    return int(RFC3526_GROUP14_P_HEX, 16), 2

# ---------- KDF & Encryption ----------
def kdf_sha256_from_K(K: int) -> bytes:
    return hashlib.sha256(K.to_bytes((K.bit_length() + 7) // 8, "big")).digest()

def xor_stream_encrypt(key32: bytes, data: bytes, nonce: bytes) -> bytes:
    out = bytearray()
    counter = 0
    offset = 0
    while offset < len(data):
        ctr_bytes = counter.to_bytes(8, "big")
        block_key = hashlib.sha256(key32 + nonce + ctr_bytes).digest()
        block = data[offset: offset + len(block_key)]
        out.extend(bytes(a ^ b for a, b in zip(block, block_key[:len(block)])))
        offset += len(block)
        counter += 1
```

```
    return bytes(out)

# ---------- State ----------
class DHState:
    def __init__(self):
        self.p = None
        self.g = None
        self.a = None
        self.b = None
        self.A = None
        self.B = None
        self.KA = None
        self.KB = None
        self.derived_key = None

# ---------- Operations ----------
def op_select_params(state: DHState):
    print("\n[1] Select / Generate Public Parameters (p, g)")
    print("    1) Use standard 2048-bit MODP group (RFC 3526 Group 14)")
    print("    2) Generate a demo safe prime (~512-bit)")
    choice = input("Choose [1/2]: ").strip()
    if choice == "1":
        state.p, state.g = get_rfc3526_group14()
        print("\nSelected RFC 3526 Group 14 (2048-bit).")
    elif choice == "2":
        print("\nGenerating a ~512-bit safe prime p = 2q+1 and subgroup generator g
...")
        p, q = gen_safe_prime(512)
        g = gen_subgroup_generator(p, q)
        state.p, state.g = p, g
        print("Generated safe prime p (~512 bits) and generator g.")
    else:
        print("Invalid choice.")
        return
    print(f"\nPublic parameters:\np = {state.p}\ng = {state.g}\nBitlength(p) =
{state.p.bit_length()} bits")

def op_generate_keys(state: DHState):
    if not state.p or not state.g:
        print("Set (p, g) first (menu option 1).")
        return
    state.a = secrets.randbelow(state.p - 3) + 2
    state.b = secrets.randbelow(state.p - 3) + 2
```

```
    state.A = pow(state.g, state.a, state.p)
    state.B = pow(state.g, state.b, state.p)
    print("\n[2] Generated keys for Alice and Bob.")
    if input("Show private exponents? [y/N]: ").lower() == "y":
        print(f"Alice private a = {state.a}\nBob private b   = {state.b}")
    print(f"Alice public A = {state.A}\nBob public B   = {state.B}")

def op_compute_shared(state: DHState):
    if not (state.A and state.B):
        print("Generate keys first (menu option 2).")
        return
    state.KA = pow(state.B, state.a, state.p)
    state.KB = pow(state.A, state.b, state.p)
    print("\n[3] Computed shared secrets.")
    print(f"K_A = {state.KA}\nK_B = {state.KB}")
    print("Verification:", "Equal" if state.KA == state.KB else " Mismatch")

def op_derive_and_demo(state: DHState):
    if not state.KA:
        print("Compute shared secret first (menu option 3).")
        return
    key = kdf_sha256_from_K(state.KA)
    state.derived_key = key
    print("\n[4] Derived symmetric key (SHA-256):")
    print(key.hex())
    msg = input("Enter plaintext: ").encode()
    nonce = secrets.token_bytes(12)
    ct = xor_stream_encrypt(key, msg, nonce)
    print("\n--- Encryption Demo ---")
    print(f"Nonce:      {nonce.hex()}")
    print(f"Ciphertext: {ct.hex()}")
    pt = xor_stream_encrypt(key, ct, nonce)
    print(f"Recovered:  {pt.decode(errors='replace')}")

# ---------- Menu ----------
def banner():
    print("\n" + "=" * 80)
    print("Experiment No. 4 - Implement Diffie-Hellman Key Exchange
Algorithm")
    print("=" * 80)
    print(textwrap.dedent("""
        Menu:
          1) Select / Generate Public Parameters (p, g)
```

```
        2) Generate Keys for Alice and Bob
        3) Compute Shared Secret
        4) Derive Symmetric Key & Demo Encrypt/Decrypt
        0) Exit
    """).strip())
    print("=" * 80)

def main():
    state = DHState()
    while True:
        banner()
        choice = input("Enter choice: ").strip()
        if choice == "1":
            op_select_params(state)
        elif choice == "2":
            op_generate_keys(state)
        elif choice == "3":
            op_compute_shared(state)
        elif choice == "4":
            op_derive_and_demo(state)
        elif choice == "0":
            break
        else:
            print("Invalid choice. Try again.")

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\nInterrupted. Exiting.")
        sys.exit(0)
```

OUTPUT:

1. Select / Generate Public Parameters (p, g) -
    a. use a standard 2048-bit MODP group

```
Enter choice: 1

[1] Select / Generate Public Parameters (p, g)
    1) Use standard 2048-bit MODP group (RFC 3526 Group 14)
    2) Generate a demo safe prime (~512-bit)
Choose [1/2]: 1

Selected RFC 3526 Group 14 (2048-bit).

Public parameters:
p (decimal) = 155251809230070893513091813125848175563133404943451431320235119490296623994910210725866945387659164244291000768028886422915080371891804634263272761303128298374438082089019628850917069131659317536746955176311984337163722100721057791 9
g (decimal) = 2
bitlength(p) = 768 bits
```

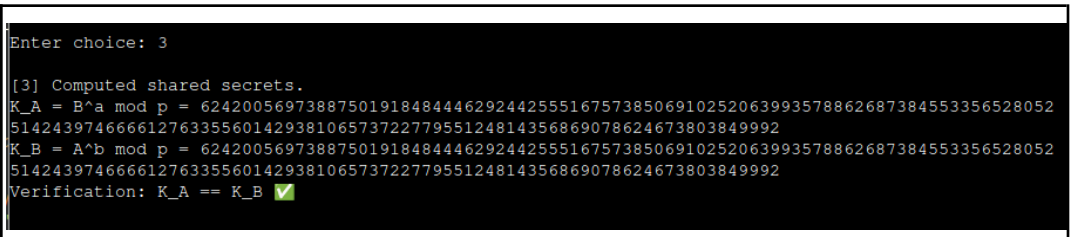
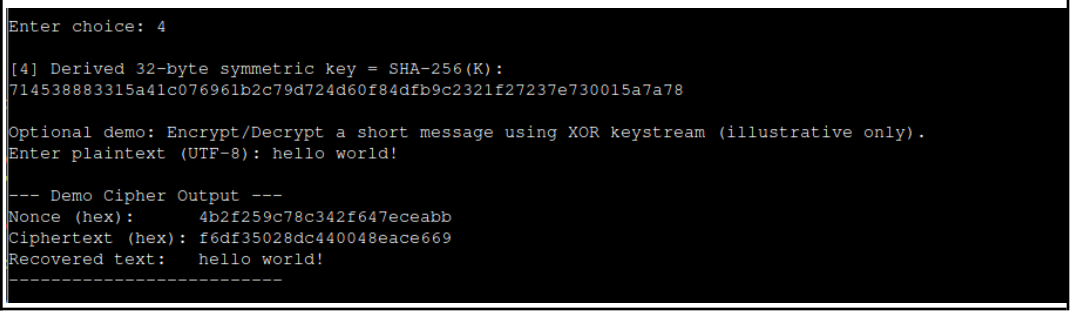
    b. Generating a demo safe prime number (choice 2):

```
Menu:
  1) Select / Generate Public Parameters (p, g)
  2) Generate Keys for Alice and Bob
  3) Compute Shared Secret
  4) Derive Symmetric Key & Demo Encrypt/Decrypt (Optional)
  5) Show Current State Summary
  0) Exit
================================================================================================
Enter choice: 1

[1] Select / Generate Public Parameters (p, g)
    1) Use standard 2048-bit MODP group (RFC 3526 Group 14)
    2) Generate a demo safe prime (~512-bit)
Choose [1/2]: 2

Generating a ~512-bit safe prime p = 2q+1 and subgroup generator g ...
Generated safe prime p (~512 bits) and generator g of order-q subgroup.

Public parameters:
p (decimal) = 11833408595567660131358809396884155735322541908098756244110358247803352481802314120031208544395350574526606566374163604601406695129300253603008179147131187
g (decimal) = 3320438367023971113142858337135690084774381520800771821165695838875512294686653506324480503203711418843081894676122795369139693760531241467791737386488514
bitlength(p) = 512 bits
```

2. Generate keys for alice and bob:

```
Menu:
  1) Select / Generate Public Parameters (p, g)
  2) Generate Keys for Alice and Bob
  3) Compute Shared Secret
  4) Derive Symmetric Key & Demo Encrypt/Decrypt (Optional)
  5) Show Current State Summary
  0) Exit
================================================================================================
Enter choice: 2

[2] Generated keys for Alice and Bob.
Show private exponents a and b? [y/N]: y
a (Alice private) = 7141024563714143067686086638549752997428090699281935086673475705926646816358385965375560976833035601262608014328913885027219877376948607698880238033390 93
b (Bob private)   = 6427244801900613998412329276413874573990253322598512618413998306261602560665659787882052612871482298332006518672762570229327577888720144743067237808383307
A = g^a mod p (Alice public) = 7843307415294137799712874103945934181357929042215975410248143286315977261611951851521778786580991600739654402157250238473051708131972015221320653081811737
B = g^b mod p (Bob public)   = 4127136757088315084338176268041466227125605030359572000821541944994386267255185723483682299697338234895319530898128246004428782505194405515396826387373080
```

3. Compute shared secrete key:

	 4. Encryption decryption of demo text: 
Conclusion:	In this experiment, we implemented the Diffie–Hellman key exchange to establish a shared secret over an insecure channel. Using public parameters (p,g) and private exponents, both parties successfully computed the same secret key. A Key Derivation Function (SHA-256) was applied to generate a symmetric key, which was used for simple encryption and decryption. The experiment demonstrated the security of DH based on the hardness of the discrete logarithm problem and highlighted attack considerations like MITM and small subgroup attacks.